

Data warehousing

Data models in database systems

Lecture 6

Mara Romanovska
Mara.Romanovska@rtu.lv

30.10.2025.



RTU
DATORZINĀTNES UN
INFORMĀCIJAS
TEHNOLOĢIJAS FAKULTĀTE

Agenda

- About PW2
- Midterm test
- Data analysis with OVER
- Modelling process and dimensions according to business logic
- Optimization
 - Aggregates
 - Column stores

PW2: Design and Implementation of a Data Warehouse

- The submission deadline is **November 30**
 - An emergency extension may be requested via email before the deadline if you have not used it before for PW1
- PW2 is a **group project** for **2-3** students
- Tasks:
 - Design the Data Warehouse
 - Implement the Warehouse and Populate Data (ETL)
 - Create Advanced Analytical Queries
 - Design an Aggregate Table
- Description published today
- Considering you have the midterm, the remaining files will be published by Monday, November 3

Changing groups?

- If group remains intact, do not do anything, it automatically transfers to this work
- If you have generated variation, you are not allowed to make changes in the group
- **If (and only if)** you want to make any changes the group and have not generated a variation, make sure to fill the form for PW2 by November 7, 18:00
- You can do that later, too (unless a variation is generated) but the process may take longer

Midterm test
November 6, 14:30



General information

- Midterm Test (worth 25%) assesses both theoretical knowledge and practical skills
- Minimum passing grade of 4 is awarded for achieving at least 50% of the available points
- **Passing the midterm examination ($\geq 50\%$) is a prerequisite for admission to the final examination**
- On-paper and on-site
- In total 50 points distributed based on how complex the task is
- The one retake of the midterm exam will be in January

Rules of the midterm test

- Absolutely no communicating
 - Absolutely no phones and looking at them for any reason
 - You are only allowed to use what is given to you
-
- Any academic integrity breach will lead to annulment of the result
 - There may be a warning depending on the seriousness of the breach

Midterm test logistics

- You will be split into 2 groups and corresponding times based on **the last three numbers of your student ID**:
 - 14:30 – students with the last three numbers (000-130)
 - 15:20 – students with the last three numbers (031-300)
- If you cannot come to the assigned time slot, you must let me know at mara.romanovska@rtu.lv before Sunday, November 2
- If you fail to come to correct time, your work may not be graded and you may get 0 for the Test 1



Topics covered

- ORDBs
- Data warehousing (including today's lecture)

Preparation

- In ORTUS, you have
 - Test material – i.e., the material that will be available to you during test
 - Sample questions
- Make sure you understand the difference between different concepts

Data analysis with OVER



Subquery vs. OVER

- OVER can be used as an alternative to the subquery in the SELECT part.

```
--alternatives using a SELECT subquery
SELECT s.driver_code, paid_salary,
       (SELECT avg(paid_salary) FROM salaries)
FROM salaries s;
```

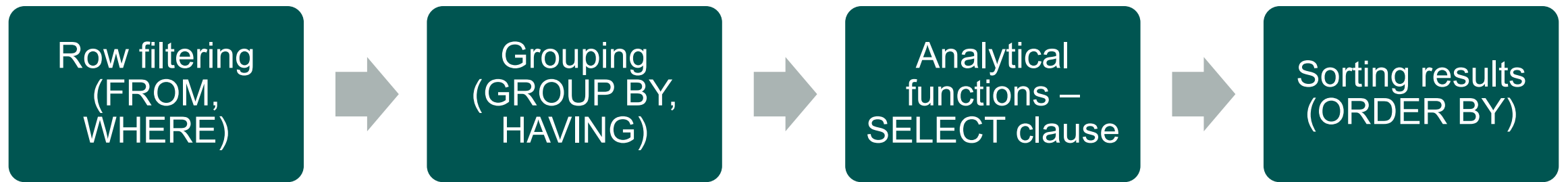
```
SELECT s.driver_code, paid_salary,
       avg(paid_salary) OVER()
FROM salaries s;
```

DRIVER_CODE	PAID_SALARY	AVG(PAID_SALARY)
67390282019	4672.25	4882.104
12436190190	5675.11	4882.104
56129018212	4660.23	4882.104
64792181931	5749.2	4882.104
98614209810	5500.72	4882.104

Using analytical functions

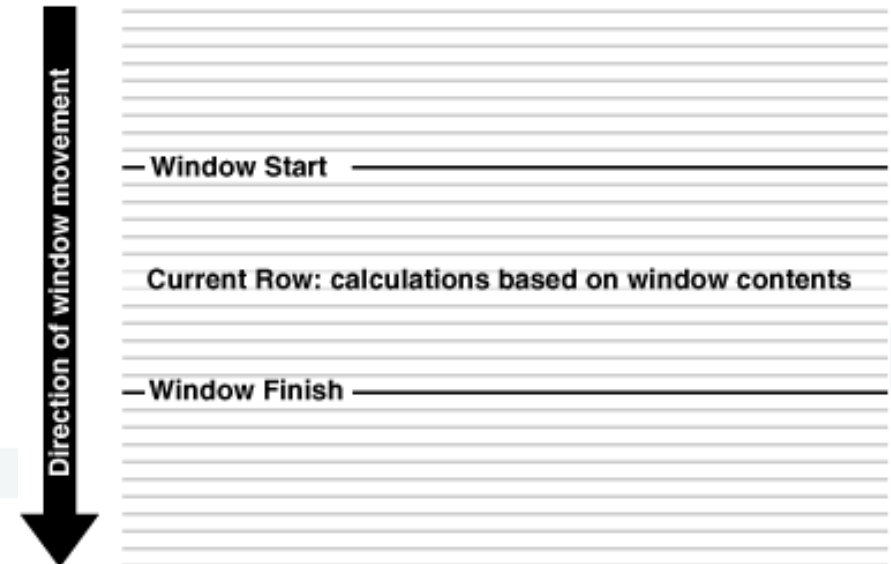
- To avoid subqueries that complicate the query and are not optimized
- On what basis are values grouped in GROUP BY?
- In addition, analytical functions allow you to work with grouping that **is not based on values**

Sequence of processing analytical queries



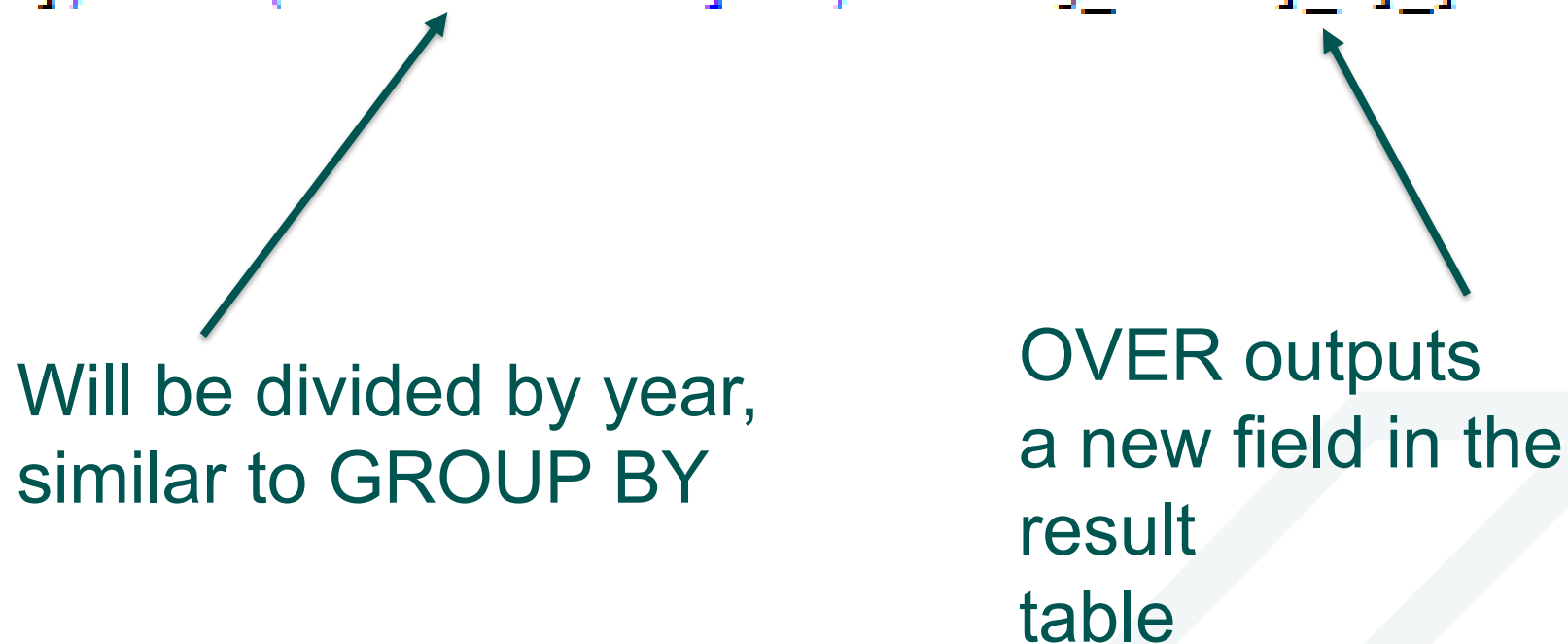
Grouping in analytical functions

- **PARTITION BY clause** – works similarly to GROUP BY
- **ORDER BY clause** is used to define the sequence for those functions that require it
- **Window creation** – a sliding window is created for each record in the clone table
 - By row numbers
 - By current row value



PARTITION BY

```
--partition by  
SELECT s.driver_code, paid_salary,  
|      avg(paid_salary) OVER(PARTITION BY year) AS avg_salary_by_year  
FROM salaries s;
```



Will be divided by year,
similar to GROUP BY

OVER outputs
a new field in the
result
table

Windowing

- The sliding window is calculated for each row
- The rows included in the window depend on
 - the active row in the table
 - the window parameter specifications
- The records in the window can be specified in the following ways:
 - specifying **a set of records**, for example, from the current record (CURRENT ROW) to the end of the table row (UNBOUNDED FOLLOWING)
 - by specifying **a range of values**, for example, all records with sales less than the sales of the current record

Window based on row numbers

--example of a sliding window (windowing)

```
SELECT s.driver_code, paid_salary,  
       avg(paid_salary) OVER(  
         PARTITION BY year  
         ORDER BY paid_salary  
         ROWS 2 PRECEDING) AS two_rows_before  
FROM salaries s;
```

Is ORDER BY
necessary?
Yes!

```
SELECT s.driver_code, paid_salary,  
       avg(paid_salary) OVER(  
         PARTITION BY year  
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS entire_table  
FROM salaries s;
```

Window based on values

--by value ranges - using RANGE

```
SELECT s.driver_code, paid_salary,  
       avg(paid_salary) OVER(  
           PARTITION BY year  
           ORDER BY paid_salary  
           RANGE 2 PRECEDING) AS specific_salary_range  
FROM salaries s;
```

A simple example of a function – ranking with RANK

```
--Ranking with RANK and DENSE_RANK  
SELECT s.driver_code, paid_salary,  
|   RANK() OVER(ORDER BY paid_salary) AS rank  
FROM salaries s;
```

**Modelling process and
dimensions according to
business logic**



Modelling process in general

- What decisions will this enable? Which metrics define success?
- Identify facts implied by questions (e.g., Sales, Claims, Tickets) and a measure
- Identify dimensions (Customer, Product, Time, Channel)

Modelling dimensions

- A slowly changing dimension (SCD) is necessary to track changes in dimensions
- Type 1: Overwrite existing dimension records.
- Type 2: Keep a full history of dimension
- Type 3: A Type 3 SCD is similar to a Type 2 SCD, but instead of creating a new row, a change in a Type 3 SCD creates a new field.

Optimization



There are various optimization techniques

- Include optimizing at data model level, query performance level, physical organization level
- Aggregate tables
- Column stores

Aggregate table



Aggregate table in general

- **Aggregate tables** are **pre-computed summary tables** that store rolled-up data (totals, averages, counts, etc.) – instead of raw transaction-level details
- Implemented as summaries (IBM); materialized views (Oracle)

Materialized views



Views – in general

- View is a dynamic virtual table that can be used for multiple purposes
- To limit access
- To simplify querying
- In other scenarios, e.g., migration

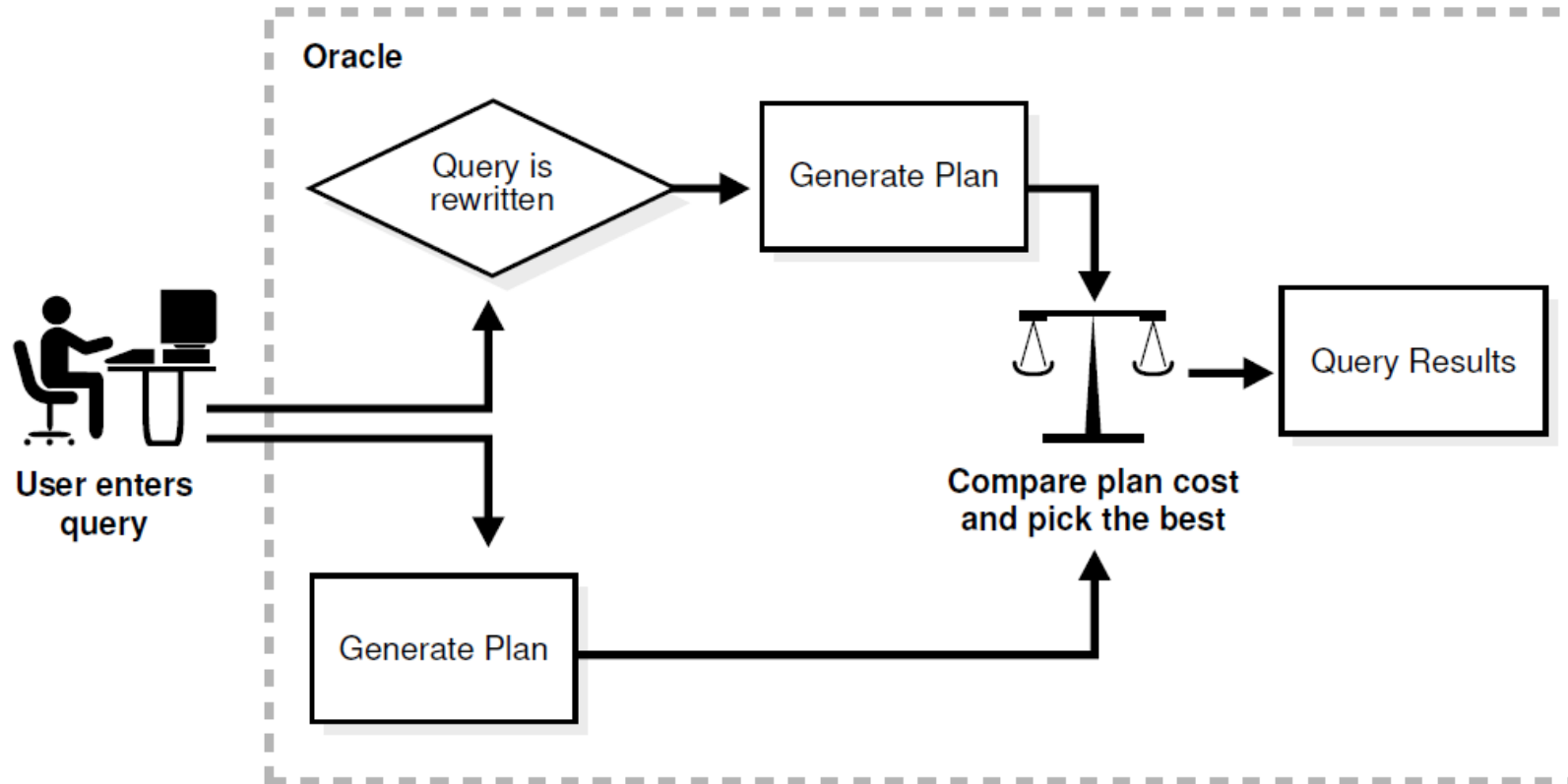
Views vs. Materialized views

- What do the views store?
- Materialized views on the other hand actually store information
- A materialized view is a precomputed table comprising aggregated and joined data from fact and possibly from dimension tables
- Materialized views are the implementation of so-called summaries

Types of materialized views

- Aggregated materialized views
- Materialized views with just the joins
- Nested materialized views

Query Rewrite



Information provided when creating materialized view

- How information is stored?
- When to fill out materialized view?
- How to update data when the data change in the tables?
- When to update data?
- Does the view is used to rewrite query
- SELECT operator

Creating a materialized view

```
CREATE MATERIALIZED VIEW cal_month_sales_mv  
BUILD IMMEDIATE  
REFRESH FORCE  
ENABLE QUERY REWRITE AS  
SELECT t.calendar_month_desc, SUM(s.amount_sold) AS dollars  
FROM sales s, times t WHERE s.time_id = t.time_id  
GROUP BY t.calendar_month_desc;
```

Indices



What is index

- Ordered table that consists of two columns
 - **Search key** (indexed value)
 - **Address**, where the row is located
- Index works just like index in a table
 - We can find a topic based on index
 - We find a page where the topic is located
 - Any index is **ordered** a certain way
 - Both are **shorter** than the book – easier to find something

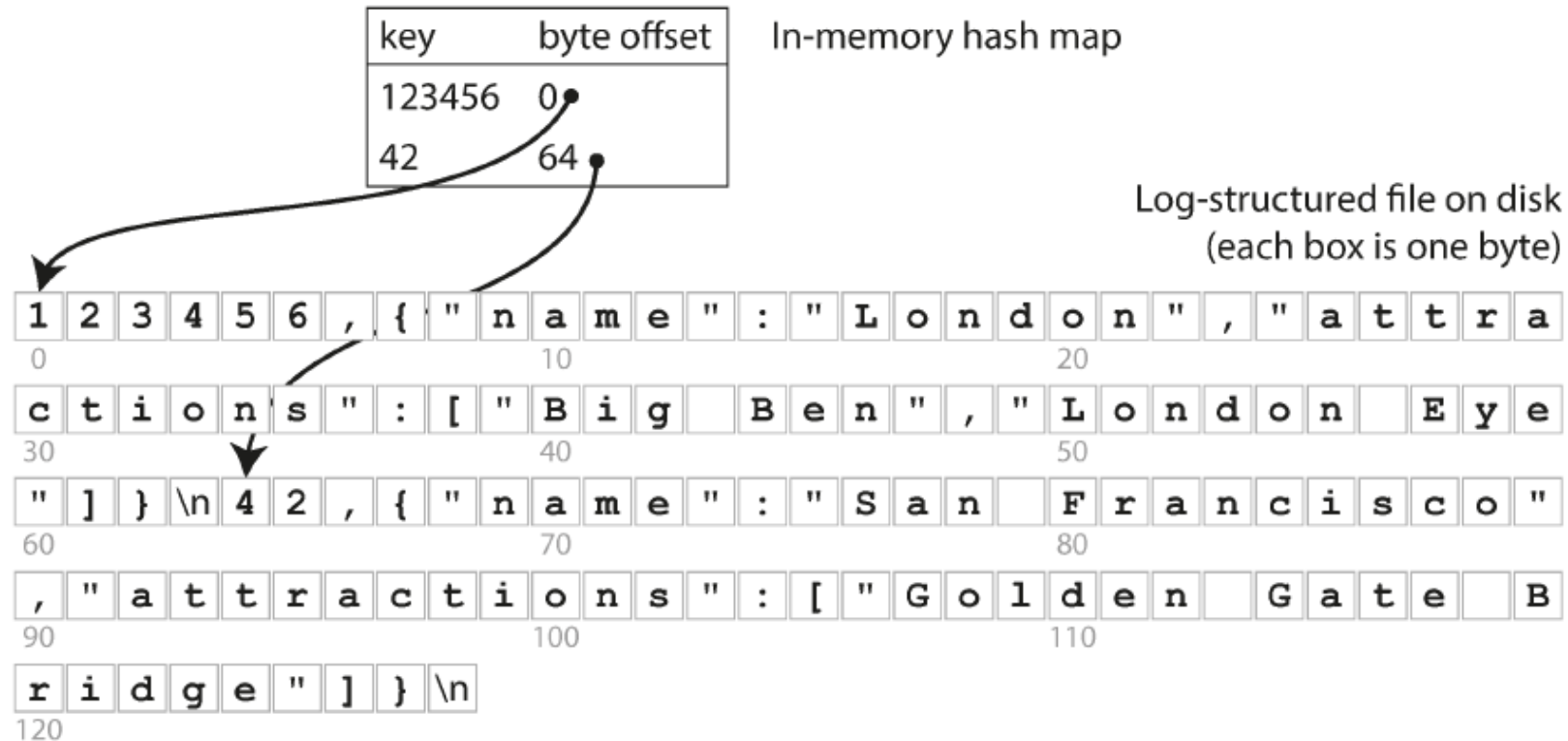
The benefits of the indices

- Indices are critical for database performance
 - Without indices, the queries would parse the tables

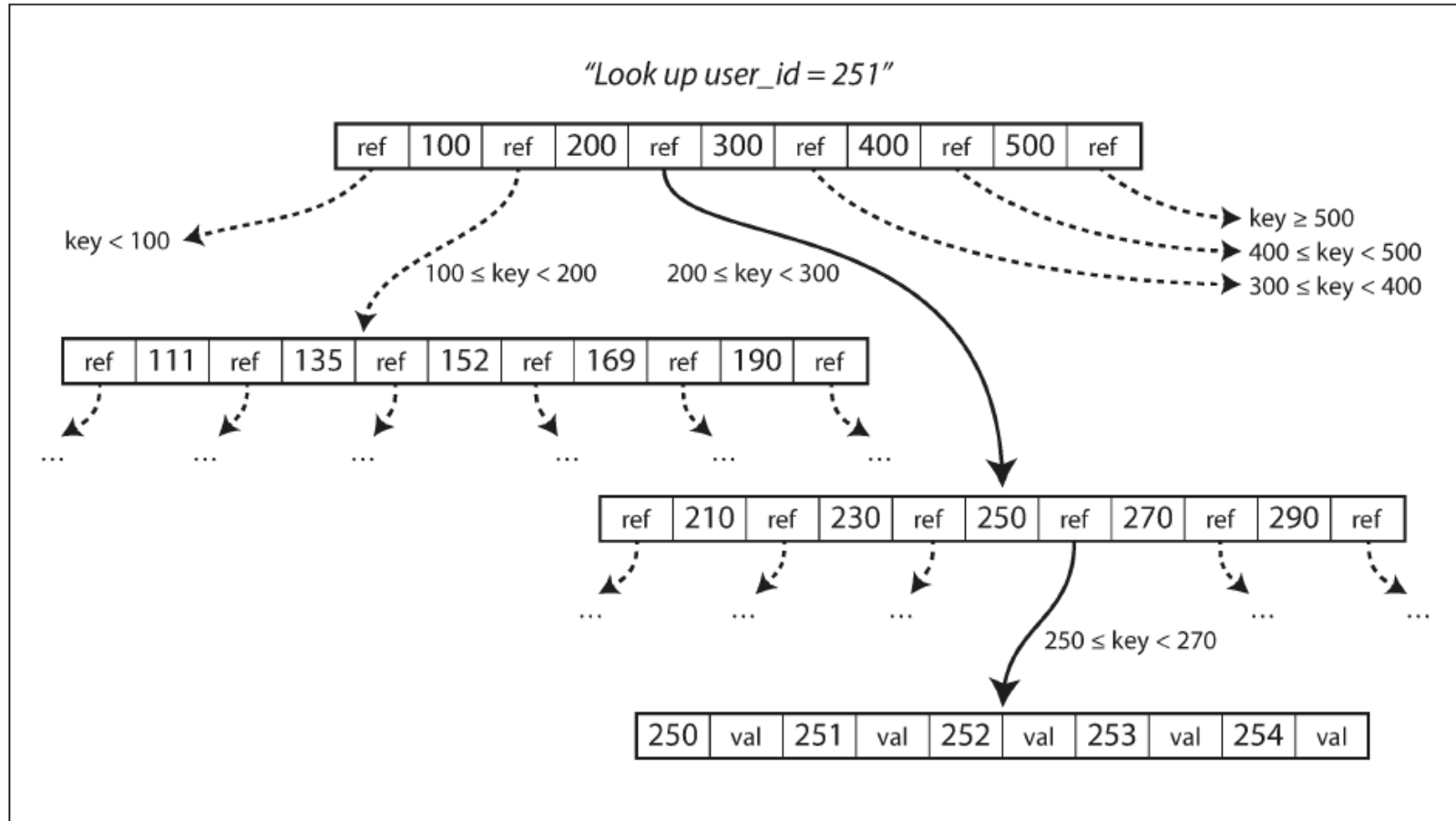
Indices are not one universal answer!

- What do we want to extract?
 - Data retrieval in intervals or specific values?
- How long data retrieval takes?
- How long does inserting data takes?
- How long does deleting data takes
- How much space the index will occupy?

Hash indices



B-tree indices



Bitmap indices

Column values:

product_sk:

69	69	69	69	74	31	31	31	31	29	30	30	31	31	31	68	69	69
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Bitmap for each possible value:

product_sk = 29:

0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 30:

0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 31:

0	0	0	0	0	1	1	1	1	0	0	0	1	1	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 68:

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 69:

1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 74:

0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Run-length encoding:

product_sk = 29: 9, 1 (9 zeros, 1 one, rest zeros)

product_sk = 30: 10, 2 (10 zeros, 2 ones, rest zeros)

product_sk = 31: 5, 4, 3, 3 (5 zeros, 4 ones, 3 zeros, 3 ones, rest zeros)

product_sk = 68: 15, 1 (15 zeros, 1 one, rest zeros)

product_sk = 69: 0, 4, 12, 2 (0 zeros, 4 ones, 12 zeros, 2 ones)

product_sk = 74: 4, 1 (4 zeros, 1 one, rest zeros)

Column-oriented stores



Column-oriented store

- Primarily used in the storage of **facts**
- Typical data warehouse query typically select just a couple of fields (columns)
- Column-oriented storage store all the values from each column together instead

The idea of column-oriented store

fact_sales table

date_key	product_sk	store_sk	promotion_sk	customer_sk	quantity	net_price	discount_price
140102	69	4	NULL	NULL	1	13.99	13.99
140102	69	5	19	NULL	3	14.99	9.99
140102	69	5	NULL	191	1	14.99	14.99
140102	74	3	23	202	5	0.99	0.89
140103	31	2	NULL	NULL	1	2.49	2.49
140103	31	3	NULL	NULL	3	14.99	9.99
140103	31	3	21	123	1	49.99	39.99
140103	31	8	NULL	233	1	0.99	0.99

Columnar storage layout:

date_key file contents: 140102, 140102, 140102, 140102, 140103, 140103, 140103, 140103
product_sk file contents: 69, 69, 69, 74, 31, 31, 31, 31
store_sk file contents: 4, 5, 5, 3, 2, 3, 3, 8
promotion_sk file contents: NULL, 19, NULL, 23, NULL, NULL, 21, NULL
customer_sk file contents: NULL, NULL, 191, 202, NULL, NULL, 123, 233
quantity file contents: 1, 3, 1, 5, 1, 3, 1, 1
net_price file contents: 13.99, 14.99, 14.99, 0.99, 2.49, 14.99, 49.99, 0.99
discount_price file contents: 13.99, 9.99, 14.99, 0.89, 2.49, 9.99, 39.99, 0.99

Utilizing column store

- **Column compression** – using bitmaps enable quicker processing, e.g., in WHERE parts of queries
- **Vectorized processing** – storing column as a vector, quickly processing as a tight array

Order of values

- Columns cannot be sorted separately
- Ordering is used for compressing columns, secondary orders may not be as efficient
- Storing data in multiple sort orders is done

Q&A

